

Good — that's enough to ground this in what Deep Agents actually gives you (not just LangChain agents with extra steps). Here's my take before I ask you anything.



The core tension you need to design around

"Complete architecture" is the risky phrase. For legacy systems, no team — human or agentic — ever produces a *ground-truth-complete* ArchiMate model; they produce a model that's complete **relative to the evidence available**, with gaps explicitly flagged. If the multi-agent system is designed to *assert* completeness rather than *bound and report* its own coverage, you'll end up with a plausible-looking model containing hallucinated elements and relationships that nobody catches until it's used for a modernization decision. So the design goal I'd actually optimize for is: **traceable, gap-aware, incrementally validated** — not "one shot, fully complete."

Recommended shape on Deep Agents

Orchestrator + specialized subagents, not one big agent. Use `SubAgentMiddleware` to split by *source type / ArchiMate layer*, not by "do everything":

- `code-analyzer` — mines repos, IaC, DB schemas → Application Components, Application Services, Data Objects, Application Interfaces
- `infra-analyzer` — CMDB, Terraform, network configs → Nodes, Devices, System Software, Technology Services
- `business-analyst` — docs, wikis, interview transcripts → Business Actors, Roles, Processes, Services
- `integration-mapper` — API specs, message queues, traffic/APM data → cross-layer flows and relationships
- `reconciler` — runs after each layer/source pass to dedupe and merge fragments into one canonical model (critical — different subagents *will* discover the same Application Component under different names)
- `validator` — checks every element/relationship has a source citation, flags orphans, and reports coverage against a checklist, rather than declaring "done"

Give the model a real ArchiMate skill. The single highest-leverage piece here: a `SKILL.md` (or several) encoding the ArchiMate 3.2 metamodel — valid element types per layer, the relationship-validity matrix, naming conventions. Without this, agents will invent plausible-but-invalid element types and relationships. This is a grounding problem, not a prompting problem — treat the metamodel as reference data agents look up, not something they recall from training.

Use the filesystem/Store backend as the model registry, not the chat transcript. Legacy-system discovery across a real enterprise system is a multi-session effort. Persist extracted elements as structured fragments (JSON/YAML with id, type, layer, name, source-evidence citation, relationships) under a `CompositeBackend` route like `/model/` → `StoreBackend`, so state survives across threads and subagent runs. `TodoListMiddleware` naturally maps to "plan by domain/layer" for the orchestrator.

Gate the merge into the canonical model with HITL. Configure `interrupt_on` for whatever tool commits a reconciled fragment into the master model — an SME should approve before an agent-inferred Business Process or Application Component becomes canonical, especially anything the model had to *infer* rather than directly observe.

Don't have agents emit ArchiMate Exchange XML directly. Have them emit the structured intermediate representation, and use a dedicated export step/tool (ideally validated against the Open Group Exchange Format schema) to serialize to XML for import into Archi or similar — LLMs are much more reliable at populating structured JSON than hand-writing valid XML.

A few things change the subagent boundaries and validation design significantly, so before I sketch this further:

Good — single system, all three layers, coverage-checklist + SME sign-off, exporting to Archi's Open Exchange XML. Here's the concrete design.

Topology

Orchestrator (the main deep agent, one long-lived thread) — owns the todo list, decides what to ingest next, runs the interview loop, triggers reconciliation/validation, and gates the final export.

Ingestion subagents (via `SubAgentMiddleware`, one per source type — not per ArchiMate layer, since a single source often yields elements for multiple layers):

- `code-analyzer` — repos, DB schemas/DDDL → Application Components, Application Services, Data Objects, Application Interfaces
- `infra-analyzer` — IaC, CMDB, network configs → Nodes, Devices, System Software, Technology Services, Artifacts
- `docs-miner` — wikis, prior diagrams, ADRs, runbooks → candidate elements/relationships across all three layers, tagged **low-confidence** since docs for legacy systems are usually stale

- `integration-mapper` — API specs, message queues, traffic data → cross-layer relationships (serving, flow, realization) that individual subagents can't see from their own source alone

Reconciler — merges fragments into the canonical model, dedupes near-duplicate elements discovered by multiple subagents under different names.

Validator — checks every element/relationship against the coverage checklist and the ArchiMate metamodel, and produces the gap report.

Exporter — deterministic tool, not an LLM call, that serializes the canonical registry to Open Exchange Format XML for Archi.

One thing to flag on the interview piece: Deep Agents subagents (the `task` tool) are **stateless single-shot calls** — they don't hold a multi-turn conversation. A live SME interview needs back-and-forth, so that belongs in the **orchestrator's own thread** (which does persist via `thread_id`), not in a subagent. Practical pattern: validator produces a gap list → orchestrator turns gaps into a targeted interview guide → a human runs the actual interview (or you pair it with a transcription pipeline) → orchestrator ingests the transcript as a normal input. Don't try to make an agent conduct the interview autonomously — SME time is the scarcest input here, so spend it closing specific gaps, not open-ended discovery.

State: the model registry, not the model in prose

Use `CompositeBackend` to route a `/model/` path to `StoreBackend`, so the registry survives across the many sessions this will realistically take. Each element is a structured record, not free text:

```
{id, layer, archimate_type, name, documentation,
 evidence: [{source_type, locator, excerpt}],
 confidence: "observed" | "inferred",
 relationships: [{target_id, type}]}
```

The `evidence` field is the whole point — an element with no citation back to code/config/doc/transcript is either flagged for review or dropped, never silently kept. This is what makes “complete” auditable instead of a black-box claim.

Ground the metamodel, don't trust recall

Build a `SKILL.md` (loaded via `SkillsMiddleware`) that encodes the ArchiMate 3.2 metamodel directly: valid element types per layer and the relationship-validity matrix (what can compose/aggregate/serve/realize/access/trigger/flow into what). Without this, agents will produce structurally plausible but invalid models — e.g. a Business Actor “realizing” a Technology Node. Give every ingestion subagent this skill; it's cheap insurance against the most common failure mode in LLM-generated EA models.

Reconciliation — don't make this pure LLM judgment

At single-system scale this is manageable, but still: use a deterministic fuzzy-match/normalization pass (name + type) to *propose* candidate duplicates, and only use the LLM to confirm merge-vs-keep-separate on the proposed pairs. Pure LLM-driven matching across dozens of elements from four independent subagents is where silent inconsistency creeps in.

Completeness = checklist + explicit gaps, not a promise

Validator's output is a checklist per layer (expected element/relationship categories — e.g. “every Application Component has ≥1 realizing Technology element and ≥1 associated Business Service”) with each item marked `covered` / `partial` / `not-applicable` / `not observed`. “Not observed” is a legitimate, visible answer — it's what keeps the system honest instead of quietly inferring an element to fill the gap. This checklist, not the raw model, is what the SME actually reviews and signs off on.

HITL — scope it to inference, not everything

Use `interrupt_on` on the `commit-to-canonical-model` tool, but split it: auto-commit directly-observed facts (e.g., a static-analysis hit showing component A calls database B), and require approval only for `confidence: "inferred"` elements/relationships and for the final export. Gating every single write will make this unusable at enterprise scale; gating inference is where the actual risk lives.

Export

Don't have any agent hand-author XML. The exporter is a plain deterministic tool: `canonical /mode1/ registry` → Open Exchange Format XML, ideally validated against the Open Group XSD before handing it to the SME for Archi import.

Want me to sketch the actual `create_deep_agent` scaffold (subagent configs, the ArchiMate `SKILL.md`, and the registry schema) as a next step, or do you want to react to this shape first?

=====

This expands scope significantly — from “one agent pipeline” to “a multi-tenant SaaS platform with a versioned modeling backend serving a 4-phase methodology.” Before I sketch the agent architecture *and* the data/backend/frontend design, a few things fork the design in incompatible directions, so I'd rather ask than guess and hand you something you have to unwind later.

One note first: ArchiMate 3.2 doesn't actually have a separate “Infrastructure” layer — Technology is the infrastructure layer (Nodes, Devices, System Software, Networks). I'll treat “Technology & Infrastructure” as one ArchiMate layer with two sub-facets unless you tell me you mean something else (e.g., a custom extension or a separate taxonomy you're layering on top).

With those decisions locked, here's the full design.

1. Multi-agent architecture (updated)

Ingestion layer — one subagent per source type, now covering the full ArchiMate metamodel

- `strategy-analyst` (*new*) — strategic plans, policy/compliance mandates, business case docs, SME interviews on drivers/goals → **Motivation** layer (Stakeholder, Driver, Assessment, Goal, Outcome, Principle, Requirement, Constraint) and **Strategy** layer (Resource, Capability, Course of Action, Value Stream)
- `business-analyst` — docs, interviews → **Business** layer (Actor, Role, Process, Function, Service)
- `code-analyzer` — repos, DB schemas → **Application** layer
- `infra-analyzer` — IaC, CMDB, network configs → **Technology & Infrastructure** layer
- `integration-mapper` — API specs, message queues, traffic data → cross-layer *and* **cross-system** relationships. This one matters more now: a tenant's systems integrate with each other, so this subagent's scope is tenant-wide, not per-system.

`reconciler` and `validator` work as before, but the `SKILL.md` grounding the metamodel now needs to cover all six layers plus the relationship-validity matrix across them — this is the piece most worth getting right early, since Motivation/Strategy elements are the ones LLMs are most prone to inventing loosely (a “Driver” or “Goal” is a soft, narrative concept — much easier to hallucinate than “this Node runs this System Software,” which is a hard fact from a CMDB record).

UML/C4 as derived views, not independent extraction

Since ArchiMate is primary, don't run separate ingestion pipelines for UML and C4 — add a `view-generator` step that projects the canonical ArchiMate model into those notations:

- ArchiMate Application Components + cross-system relationships → **C4 Context/Container** diagrams (system landscape, per-system container view)
- ArchiMate Application Components/Interfaces → **C4 Component** diagrams and **UML Component** diagrams
- ArchiMate Nodes/Devices/Artifacts → **UML Deployment** diagrams

- Data Objects + their structure → **UML Class** diagrams (where the source evidence supports that level of detail — e.g., an actual schema, not an inferred one)

This should be a *documented mapping table* (another grounding skill: “ArchiMate → C4/UML projection rules”), executed largely deterministically. Reserve LLM judgment for the one genuinely non-mechanical call: **what belongs at each C4 zoom level** (Context vs. Container vs. Component) — that’s a curation decision, not a translation. Keep the projection auditable: every UML/C4 element carries a back-reference to the ArchiMate element(s) it was derived from, so a reviewer can always trace a C4 box back to the As-Is fact that justified it.

The four phases as git-backed stages, not four separate pipelines

Each phase’s output is a commit/tag in the tenant’s model repo, and each phase explicitly reads a *pinned version* of the previous phase’s output — never “the latest state,” so a Gap analysis always references an exact As-Is baseline, immune to it changing underneath it later.

- **Phase 1 — As-Is:** the pipeline above. Output: tagged baseline commit (e.g. `as-is/v1`).
- **Phase 2 — Assessment (133 dimensions / 9 perspectives):** this is a fan-out problem, well suited to the `swarm` pattern — but fan out **per perspective (9 rows)**, not per dimension (133 rows). Each perspective-assessor subagent (strategy, business, application, UX, technology, infrastructure, operation, integration, security) scores all of its own dimensions against the pinned As-Is baseline plus any perspective-specific evidence (e.g. security needs scan results/pentest data, operations needs incident/monitoring history — flag now that some perspectives need input sources beyond what Phase 1 gathers). Each dimension record: `{dimension_id, perspective, score, rationale, evidence_refs[], confidence}`. Output: tagged `assessment/v1`, referencing `as-is/v1`.
- **Phase 3 — Gap vs. Target:** needs a Target architecture as input. I’m assuming for now that Target is **human-authored with agent assistance** (agents propose target-state patterns informed by assessment gaps and modernization reference architectures, e.g. cloud-native/microservices patterns) rather than fully agent-generated — flag this back to me if that’s wrong, since it changes whether Phase 3 needs its own “target-drafting” subagent loop or just a comparison engine. A `gap-analyzer` diffs As-Is vs. Target element-by-element and dimension-by-dimension, producing gap records with effort/complexity estimates, each linked to the assessment dimension and target element that motivated it.
- **Phase 4 — Roadmap:** a `roadmap-planner` subagent consumes gap records + the cross-system integration graph (sequencing constraint — you can’t retire System A’s interface before System B’s replacement for it exists) and drafts initiatives tagged with a modernization strategy (Rehost/Replatform/Refactor/Rearchitect/Rebuild/Replace/Retain). This is the highest-stakes, most strategic output — heaviest HITL gate of the four.

2. Data & storage design

Two-tier split, consistent with silo isolation but avoiding one-schema-per-client-for-everything overkill:

- **Control plane (shared, low-sensitivity):** tenant registry, users/roles, billing, job orchestration state, the **133-dimension catalog** itself (the methodology is the same for every client — only the *scores* are tenant data), audit log index.
- **Tenant silo (per client):** a Postgres **schema-per-tenant** by default (promote a client to a fully separate DB instance if their contract/compliance posture demands it — schema-per-tenant is the practical default at “dozens of clients” scale; a hard isolation requirement for a specific client is a per-client override, not the baseline).

The model itself lives in git, not as rows. Concretely: one git repo per tenant, with a directory per legacy system plus a `/landscape/` directory holding cross-system integration elements. Deep Agents’ `FilesystemBackend / CompositeBackend` gives the agent session a working checkout; a deterministic step (not the LLM) does `git add/commit` to a feature branch per extraction or assessment run, then opens a PR. **This maps your HITL approval gates directly onto PR review** — reuse an existing git server’s (e.g. Gitea/Gitness, self-hosted for tenant isolation) diff/review/merge tooling instead of building bespoke approval UI from scratch. Merged commits on `main` are the canonical, versioned baselines that later phases pin against.

The tenant’s relational schema then holds a **queryable index**, not the authoritative content:

- `legacy_systems`, `system_integrations` (structured mirror of the cross-system graph, for fast querying)
- `model_element_index` (`element_id`, `system_id`, `layer`, `archimate_type`, `name`, `git_path`, `current_commit`) — full element detail (evidence, docs, relationships) stays in the git JSON files; this table is just for search/filter/reporting without parsing git on every request
- `assessment_runs`, `assessment_scores` (referencing `dimension_catalog` in the control plane)
- `target_architectures`, `gap_records`
- `roadmap_initiatives` (with modernization-strategy tag and dependency links)
- `artifact_versions` (mirrors git log — commit sha, phase, tag, author, approval status — for fast UI queries)

Raw evidence (uploaded docs, transcripts, scan reports, rendered diagram images/XML) goes in object storage under a per-tenant prefix or bucket.

3. Backend

- **Tenant resolution must be structural, not request-supplied:** derive the DB schema connection and git repo path solely from the authenticated tenant claim in the JWT — never from a tenant ID the client sends — so there's no code path where a bug could cross tenant boundaries.
- **Async job orchestration:** agent runs (ingestion, 9-perspective assessment fan-out, roadmap drafting) are long-running — put them behind a durable task queue (Celery/Temporal or similar) with per-job status in the control-plane DB, and stream progress to the frontend via SSE/websockets rather than polling.
- **RBAC scoped per tenant:** at minimum, client SME/reviewer (approves PRs, answers interview prompts), client admin, and platform/consultant admin roles — approval routing must resolve to the correct human within the correct tenant.
- **Audit everything:** every agent-initiated commit, every human approval/rejection, every cross-phase read — government clients will ask for this in an audit, and it's much cheaper to build in from day one than retrofit.

4. Frontend

- **Tenant workspace** with a system switcher (multi-system tenants) and a **phase navigator** that mirrors the git branch/tag structure (As-Is → Assessment → Gap → Roadmap), so users always know which pinned baseline they're looking at.
- **Model viewer:** layered ArchiMate views plus derived C4/UML views, with an element detail panel surfacing evidence citations — traceability needs to be one click away, not buried in a JSON blob.
- **Diff/review UI:** since versioning is git-backed, this can literally be a diff view between two tags/commits (elements added/removed/changed) doubling as the PR-review surface for HITL approval.
- **Assessment dashboard:** 133-dimension scorecard grouped by the 9 perspectives, heatmap/radar visualization, drill-down to rationale + evidence per dimension.
- **Gap & roadmap views:** gap register linked back to assessment dimensions and target elements; roadmap as a dependency-aware timeline, initiatives tagged by modernization strategy — the cross-system dependency links from `system_integrations` should visibly constrain what can be sequenced when.

Open items I'm flagging rather than blocking on

1. **Target architecture authoring** — I assumed human-authored-with-agent-assist; confirm or correct, since it determines whether Phase 3 needs its own drafting loop.
2. **Perspective evidence gaps** — UX, operations, and security assessment dimensions likely need input sources (usability data, monitoring/incident history, scan/pentest results) that Phase 1's ingestion subagents don't collect. Worth listing what's actually available per perspective before building the assessor subagents.
3. **Git hosting choice** — self-hosted Gitea/Gitness per the isolation requirement, vs. some other git-hosting approach — affects how directly you can reuse PR tooling for the HITL gates.

Want me to proceed to a concrete scaffold next (subagent configs, the ArchiMate+projection `SKILL.md`s, the tenant repo layout, and the relational schema DDL), or react to this shape first?

=====

Got it — this refines the design meaningfully. Quick synthesis of what changes, then three things I need to pin down before sketching anything.

Target architecture (Phase 3) — this becomes a genuine multi-round negotiation among discipline subagents, mirroring how a real consulting team works:

- *Layer-owning architects* draft target-state elements for their layer: `enterprise-architect` (chief — Motivation/Strategy, sets goals/principles/constraints that bound everyone else), `business-architect` (Business), `application-architect` (Application), `infrastructure-architect` (Technology & Infrastructure), `integration-architect` (cross-system relationships/interfaces), with `solution-architect` and `ux-architect` bridging/refining the Application layer.
- *Cross-cutting reviewers* — `security-architect`, `data-architect`, `devsecops-architect` — don't own a layer; they review and annotate every other discipline's draft, since security/data/ops are cross-cutting concerns in real EA practice, not silos.

- The `enterprise-architect` runs synthesis: collect drafts → collect cross-cutting annotations → surface conflicts → targeted re-draft only from the disciplines in conflict → repeat until stable → human sign-off gate before the Target baseline is tagged.
- Each discipline subagent gets its own `SKILL.md` (reference architectures/patterns/standards for that discipline), loaded on demand — same `SkillsMiddleware` mechanism as the `ArchiMate` metamodel skill, just discipline-scoped.

Assessment (Phase 2) — this confirms and simplifies something: the ingestion subagents (`strategy-analyst` , `business-analyst` , `code-analyzer` , `infra-analyzer` , `integration-mapper`) are a **shared, reusable service**, not Phase-1-only. Phase 2 becomes: identify evidence gaps against a dimension's rubric → re-invoke the *same* ingestion subagents with dimension-scoped instructions (including new interviews) → score using a rubric-specific skill. I'll design the skills at perspective granularity (9 files) by default, but that's easy to adjust once I see your Excel sheet's actual structure — send it whenever's convenient.

One design point this raises, worth confirming: when Phase 2 evidence *changes* the As-Is model (e.g. a new interview surfaces a component nobody found in Phase 1), I'd rather that create a new As-Is version (`as-is/v2`) that the assessment then pins to — one continuously-refined source of truth with full version history — rather than a separate parallel “assessment evidence” store that could drift from the official As-Is baseline. Flag if you had something else in mind.

Git hosting — I'll design a small internal adapter interface (create branch, commit, open PR, diff, merge, read file-at-ref) with two implementations selected per tenant: a self-hosted git server (e.g. Gitea/Gitness for the on-prem case, giving you real PR/review tooling) and the GitHub API (cloud case). Tenant config records which backend + credentials it uses.

Three things I need before designing further:

One more fork worth resolving before I sketch anything, because “entire platform per client” for on-prem changes something fairly fundamental.

Full on-prem deployment means the backend, DB, agents, and git all need to run inside the client's environment as a deployable unit (containerized, config-driven for cloud vs. on-prem — object storage abstracted so it works against your cloud provider or an on-prem equivalent like MinIO, same for the git adapter we already planned). That part's mechanical.

The part that isn't mechanical: **which LLM the Deep Agents actually call**. If a government client requires full on-prem specifically for data sovereignty, routing their legacy-system evidence through a cloud LLM API (Anthropic, OpenAI, etc.) undercuts the reason they wanted on-prem in the first place. This affects the core agent runtime design, not just infra — it determines whether the discipline-architect negotiation, `ArchiMate` validity checking, and 133-dimension scoring can rely on frontier-model reasoning everywhere, or need to be designed to also work against a smaller self-hosted model for some clients.

Good — that's the right default. I'll design the agent runtime with a swappable model-provider layer (Deep Agents' model parameter takes any LangChain-compatible chat model, so this is a config choice per tenant, not a rewrite) and flag per-client model selection as a deployment-time decision, not an architecture one.

That closes the last open fork. Here's where things stand across all three rounds of clarification:

1. **Target architecture** — multi-round discipline-architect negotiation (layer owners + cross-cutting reviewers), bounded auto-negotiation with escalation to a human on unresolved conflicts, chief `enterprise-architect` synthesizes.
2. **Assessment** — reuses Phase 1's ingestion subagents (not a separate pipeline), evidence gaps trigger re-ingestion + new interviews, dimension scoring via perspective-scoped rubric skills (pending your Excel sheet), new evidence versions the As-Is baseline forward rather than forking it.
3. **Git hosting** — adapter interface supporting both self-hosted git server and GitHub, selected per tenant.
4. **Deployment** — full platform (DB, agents, compute, git) must be packageable for on-prem per client, not just cloud SaaS — everything abstracted behind swappable providers (storage, git, and now model).

That's a coherent, buildable picture now. Before I sketch the concrete architecture (subagent roster with skills, tenant repo layout, relational schema, deployment topology for cloud vs. on-prem) — is there anything else you want to pin down first, or should I go ahead and produce that scaffold?

=====

This one follows the same pattern as git/storage/model: it needs to be **pluggable per tenant**, because traces contain the same sensitive evidence (code excerpts, interview content, model data) as everything else — you can't default every client to a shared cloud LangSmith project once you've committed to per-tenant isolation and full on-prem for some clients.

Recommended shape

An observability-provider abstraction, mirrored to the git/storage/model config already in the tenant record:

- **LangSmith** (cloud, or LangSmith Self-Hosted for enterprise clients who want SaaS-grade tooling but can't send data off-network) — the natural default for cloud-hosted tenants. It's built for exactly this shape of system: LangGraph/Deep Agents runs produce **nested run trees** with essentially zero extra code (just tracing env vars), so a full ingestion run, a 9-way discipline-architect negotiation, or a 133-dimension assessment fan-out all show up as a parent run with every subagent call nested underneath — which is exactly the view you need to debug “why did the security-architect and data-architect conflict in round 2.” It also ships dataset/evaluator tooling for regression testing (see below), which you'd otherwise have to build.
- **Langfuse (self-hosted)** as the primary open-source alternative for fully air-gapped on-prem clients — purpose-built for LLM observability specifically (traces, cost/token tracking, prompt management, evals), genuinely self-hostable, and has a LangChain callback integration so the switch is config, not a rewrite. I'd reach for this before a raw OpenTelemetry stack unless a client already mandates their own observability tooling.
- **Raw OTEL → Grafana Tempo/Prometheus/Loki** as the fallback when a client's on-prem environment already has an observability stack you need to plug into rather than stand up your own.

Selection lives in the same tenant config as the git/storage/model provider choice.

What’s bespoke regardless of backend

None of these tools know about your domain — the link between “this trace” and “this git commit” and “this PR approval” is something you build yourself: tag every subagent invocation with {tenant_id, system_id, phase, run_id}, and record run_id alongside the resulting git commit sha in artifact_versions. That's the join that lets a reviewer go from a model element → the exact agent execution and evidence that produced it → whether and by whom it was approved. That audit trail matters more here than in a typical app, given the government-client, compliance-adjacent context.

What specifically needs watching, given this system’s shape

- **Fan-out visibility** — the 9-discipline Target negotiation and the 9-perspective (or 133-dimension) assessment scoring are exactly what nested run trees are good at showing; make sure every subagent call carries its round number and discipline/dimension as trace metadata/tags so you can filter “show me round 2 of the target negotiation for tenant X.”
- **Negotiation round counts and escalation rate** — since you chose bounded auto-negotiation with escalation, tracking how often disciplines actually converge within the cap (vs. hitting it and escalating) tells you whether that bound is well-tuned.
- **Per-tenant cost/token rollups** — tag traces by tenant/system/phase so cost can be attributed per client engagement, which you'll likely want for billing regardless of observability tool.
- **HITL wait time / approval funnel** — this one is *not* an observability-tool concern; it's a product metric (how long does a PR sit waiting on a client SME) that belongs in your own control-plane DB and product dashboard, not LangSmith/Langfuse.

Related but distinct: eval/regression testing

Worth calling out separately since it uses the same tooling but isn't runtime monitoring: build golden-example datasets for the correctness-critical pieces — ArchiMate relationship validity, dimension scoring against a rubric, conflict detection in Target negotiation — and run them as regression evals (LangSmith's or Langfuse's dataset/evaluator features) whenever you touch a skill or prompt. These are the places a silent regression is most expensive — a wrong score or an invalid model is worse than a slow response.

That covers observability. Between this and the four earlier rounds, I think the requirements are now complete enough to sketch the concrete architecture — subagent roster with skills, tenant repo layout, relational schema, and deployment topology for cloud vs. on-prem, including the provider abstractions (git, storage, model, observability). Want me to go ahead and produce that now?

=====

I've written the full design to [architecture-design.md](#). Summary of what's in it:

- **29 subagents** cataloged across shared ingestion services, 9 perspective-assessors, 10 discipline architects (7 layer-owners + 3 cross-cutting reviewers) plus chief synthesis, and roadmap planning — with the Phase 3 negotiation protocol laid out as a sequence diagram (bounded rounds → escalation).
- **Skills catalog** — ArchiMate metamodel, C4/UML projection rules, per-perspective rubrics, per-discipline reference architectures, modernization taxonomy.
- **Tenant git repo layout** with the full tag/branch scheme tying every phase to a pinned, human-approved commit.
- **DB schema** split control-plane vs. tenant-silo, including `artifact_versions` as the bespoke join between traces, commits, and approvals.
- **Provider abstraction table** (git/storage/model/observability) — cloud default vs. on-prem/air-gapped option for each.
- **Backend and frontend module breakdowns**, deployment topology diagram, and a **suggested MVP-first build sequence** (8 stages, control plane → As-Is → multi-tenancy → projections → Assessment → observability → Target/Gap → Roadmap → on-prem packaging) so this doesn't need to be built all at once.
- Three **open items** flagged at the end (rubric sheet, per-client model hosting, on-prem ops model) rather than blocking on them.

Next natural step, whenever you're ready, would be picking one slice from the build sequence — most likely the Phase 1 `create_deep_agent` scaffold with the ArchiMate metamodel skill — to start implementing against.

=====
